

控制流

Julia基础语法

苏州同元软控信息技术有限公司版权所有，
未经授权，不得复制、传播或以其他方式使用；
侵权必究。

苏州同元软控信息技术有限公司

2024年2月27日



TONGYUAN
Software and Control

目录

1. 复合表达式

2. 逻辑判断

3. 循环表达式

1. 复合表达式-最简单的代码块

复合表达式构造：多个语句变为一个语句

举个简单的例子

```
julia> a=5*8;b=a^2
```

```
1600
```

这时，Julia会从左到右逐个对这些表达式求值，并把**最后一个表达式的结果值作为最终的结果**。

像上面这样的并列的表达可以称为代码块。

并列表达式可以作为一个整体出现在赋值符号“=”右边

```
julia> c1=(a=5*8;b=a^2)
```

```
1600
```

```
julia> c1
```

```
1600
```

- ① 子表达式;子表达式;
- ② (子表达式;子表达式;子表达式.....)

这就相当于我们把这个并列表达式的最终结果值赋给了变量c1。

注意：这里的圆括号**不能省略**，否则会引起歧义。

1. 复合表达式-最简单的代码块

我们还可以使用关键字 `begin` 和 `end` 对并列表达式进行包装，以形成边界更加清晰的代码块。

(`begin-end`):

`begin`

#任意多的子表达式

`end`

```
julia> begin a=5*8;b=a^2 end
1600

julia> c2=begin a=5*8;b=a^2 end
1600

julia> c2
1600
```

说明:
Julia中不要求将分号作为语句结尾字符;
多个表达式在一行中时, 需用分号隔开且不可省略

下述代码块也称为 `begin` 代码块。
`begin` 代码块的优势在于, 当并列表达式中的子表达式过多时, 我们可以分成多行来编写, 其边界依然清晰。

```
julia> c2=begin
           a=5*8
           b=a^2
        end
1600
julia>                                     begin
```

```
julia> c2=(a=5*8;b=a^2)
1600

julia>                                     并列
```

无论是并列的表达式, 还是`begin`代码块, 我们都称之为“复合表达式”。
主体只是多个子表达式的排列, 没有任何逻辑处理, 所以是最简单的代码块。

目录

1. 复合表达式

2. 逻辑判断

3. 循环表达式

2. 逻辑判断

if语句:

一般格式:

```
if 条件表达式1
    #执行语句
elseif 条件表达式2
    #执行语句
elseif 条件表达式3
    #执行语句
else
    #执行语句
end
```

注意:

- 条件表达式中if是必须的, 且只能出现 1 次;
- else 可以不出现但若出现则只能 1 次;
- elseif 可多次使用;
- 条件表达式与关键词之间用空格隔开
- 条件表达式为布尔量

```
julia> a=10
    if a>0
        3
    elseif a<0
        2
    end
3

julia> if 1 #条件表达式只能为布尔量
    println("true")
end
ERROR: TypeError: non-boolean (Int64) used in boolean context

julia> z = if 3 < 0 #if语句也属于复合表达式
    10
    else
        -10
    end
-10
```

说明:

- if语句中只有一个分支会被执行
- 不支持将0或1作为true或false使用
- if语句也属于复合表达式, 可以将其赋给一个变量

2. 逻辑判断

三元运算符:

一般格式: `a ? b : c`



功能等价

说明:

- 各符号之间有空格;
- a是条件表达式, 值为true时该三元运算符返回b值, 否则返回c值;

```
if a
    b
else
    c
end
```

```
julia> x=3; x>10 ? true : false
false
```

```
julia> 3>0 ? 3 : 0
3
```

```
julia> false ? 3 : 0
0
```

2. 逻辑判断

短路运算(&&与||): 表达式只要能够确认最终结果，就会提前返回且不执行后续的判断

一般格式:

- `a && b`,子表达式b仅当a为true时被执行
- `a || b`,子表达式b仅当a为false时被执行

```
julia> action="walk";weather="rain";prompt=""  
##定义了变量action和weather和prompt  
  
if action=="walk"&&weather=="rain"  
    prompt="Don't forget to bring an umbrella"  
end  
"Don't forget to bring an umbrella "
```

```
julia> action="walk";weather="rain";prompt=""  
##定义了变量action和weather和prompt  
  
if action=="sleep"&&weather=="rain"  
    prompt=" Don't forget to bring an umbrella "  
else prompt=" Don't bring an umbrella "  
end  
"Don't bring an umbrella "
```

action的值由 “walk” 修改为 “sleep” if语句中的第二个条件就没有求值的机会了，因为第一个条件的求值结果为false，从而造成 “短路”



2. 逻辑判断

短路运算(&&与||): 表达式只要能够确认最终结果, 就会提前返回且不执行后续的判断

一般格式:

- `a && b`,子表达式b仅当a为true时被执行
- `a || b`,子表达式b仅当a为false时被执行

```
julia> action= "walk";weather="rain";prompt=""  
##定义了变量action和weather和prompt  
  
if action== "sleep" && weather=="rain"  
    prompt=" Don't forget to bring an umbrella "  
else prompt=" Don't bring an umbrella "  
end  
"Don't bring an umbrella "
```

```
julia> action= "walk";weather="rain";prompt=""  
##定义了变量action和weather和prompt  
  
if action== "sleep" || weather=="rain"  
    prompt=" Don't forget to bring an umbrella "  
else prompt=" Don't bring an umbrella "  
end  
"Don't bring an umbrella "
```

条件的符号由 “&&” 修改为 “||” , “||” 也是 “**短路**” 条件求值, 但是它代表的是 “或者” 。

Julia会从左到右依次判断表达式的每一个条件, 但是只要发现当前结果为true, 就不会继续判断了, 直接输出true。只有所有的条件都是false, 最终的结果才是false。

目录

1. 复合表达式

2. 逻辑判断

3. 循环表达式

3. 循环表达式

while语句

一般格式:

```
while 条件表达式
#执行语句
end
```

```
julia> i=1
1

julia> while i <= 5
println(i)
global i += 1
end

1
2
3
4
5
```

```
julia> i=1
1

julia> while i <= 5
println(i)
i += 1
end

1
2
3
4
5
```

不建议这样书写
Ctrl+Enter

说明:

- 条件表达式为布尔量，为true时，执行语句才会执行;



3. 循环表达式

for语句:

一般格式(单个迭代变量):

格式1:

```
for 迭代变量=可迭代数集  
    #语句块  
end
```

格式2:

```
for 迭代变量 in 可迭代数集  
    #语句块  
end
```

格式3:

```
for 迭代变量 ∈ 可迭代数集  
    #语句块  
end
```

注意:

- 三个遍历操作符=、in及∈是等价的，可以任选一个；
- ∈输入：`\in + tab`键
- 可迭代的内容包括元组、字典、集合、数组、及列表等

```
julia> for i = 1:5  
    print(i)  
end  
12345  
  
julia> for i in [1 2 3 4 5]  
    print(i)  
end  
12345  
  
julia> for i ∈ (1, 2, 3, 4, 5)  
    print(i)  
end  
12345
```

3. 循环表达式

多个迭代变量:

```
julia> for i = 1:2, j in (1, 2, 3)
    print((i, j))
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)
```

```
julia> for i in [1 2 3], j = 3:4
    print((i, j))
end
(1, 3)(1, 4)(2, 3)(2, 4)(3, 3)(3, 4)
```

```
julia> for j = 3:4, i in [1 2 3]
    print((i, j))
end
(1, 3)(2, 3)(3, 3)(1, 4)(2, 4)(3, 4)
```



```
julia> for i = 1:2
    for j in (1, 2, 3)
        print((i, j))
    end
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)
```

```
julia> for i=1:2;j=3:4
    print((i,j))
end
(1, 3:4)(2, 3:4)
```

说明:

- for循环支持同时对多个数据集进行遍历;
- 遍历操作符的顺序影响内部语句循环变量的取值。

注意:

- 每一个遍历表达式之间的分割符是逗号而非分号, 当为分号时, 这种表达式将为复合表达式而非遍历表达式。



3. 循环表达式

多个迭代变量:

```
julia> for i = 1:2, j in (1, 2, 3)
    print((i, j))
    i=2
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)
```



```
julia> for i = 1:2
    for j in (1, 2, 3)
        print((i, j))
        i=2
    end
end
(1, 1)(2, 2)(2, 3)(2, 1)(2, 2)(2, 3)
```

在右侧的执行结果中，第二个元组和第三个元组里面的第一个元素值都变成了2，而不是原来的1，这就是这个代码*i=2*在内层for语句中起到的作用。

当嵌套的循环被合并在一起时，其中的迭代变量的值就必定不会受到任何干扰，它们只取决于对应的被迭代对象和当时的迭代状态。
而当嵌套的循环是由多条 for 语句松散地表达时，上述干扰就很容易发生。



3. 循环表达式

continue—强制跳过当前迭代

```
julia> i=1
1

julia> while i < 10
    i += 1
    if i < 5
        continue
    end
    print(i)
end
5678910

julia> i
10
```

说明：

- continue可以让循环跳过当前迭代，直接进入下一个循环的迭代。
- continue在for循环中的使用与while循环相同。

```
julia> i=1;j=1;

julia> while j <= 4
    while i < 10
        i += 1
        if i < 5
            continue
        end
        print(i)
    end
    j += 1
    i = 1
    println()
end
5678910
5678910
5678910
5678910
```

说明：

- continue在多层嵌套的循环中只会影响其所处的那一层，不会波及外层或内层循环结构。

3. 循环表达式

break—强制退出循环

```
julia> i=1
1

julia> while i < 10
    if i > 5
        break
    end
    print(i)
    i += 1
end
12345
```

说明：

- break强制循环在满足控制条件时退出。
- break在for循环中的使用与while循环相同。

```
julia> i=1;j=1;

julia> while j <= 4
    while i < 10
        print(i)
        if i > 5
            break
        end
        i += 1
    end
    j += 1
    i = 1
    println()
end
123456
123456
123456
123456
```

说明：

- break在多层嵌套的循环中只会影响其所处的那一层，不会波及外层或内层循环结构。

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

请各位专家指正！

苏州同元软控信息科技有限公司版权所有，侵权必究。
传播或以其他方式使用；